

```
<!--IIC2513-->
```

JS en Profundidad

{

```
<Tema="Asincronía & Event Loop"/>
```

```
<Por="Vicente Ignacio Soto González"/>
```

}



Problema del Bloqueo

Modelo de un Solo Hilo:

JavaScript ejecuta una sola instrucción a la vez en su único Call Stack. No cuenta con múltiples hilos nativos en el motor para procesar scripts paralelos.

```
console.log('inicio');

// Simula una tarea pesada bloqueante
const t0 = Date.now();
while (Date.now() - t0 < 3000) {}

console.log('fin');
```

Colapso de la Interfaz:

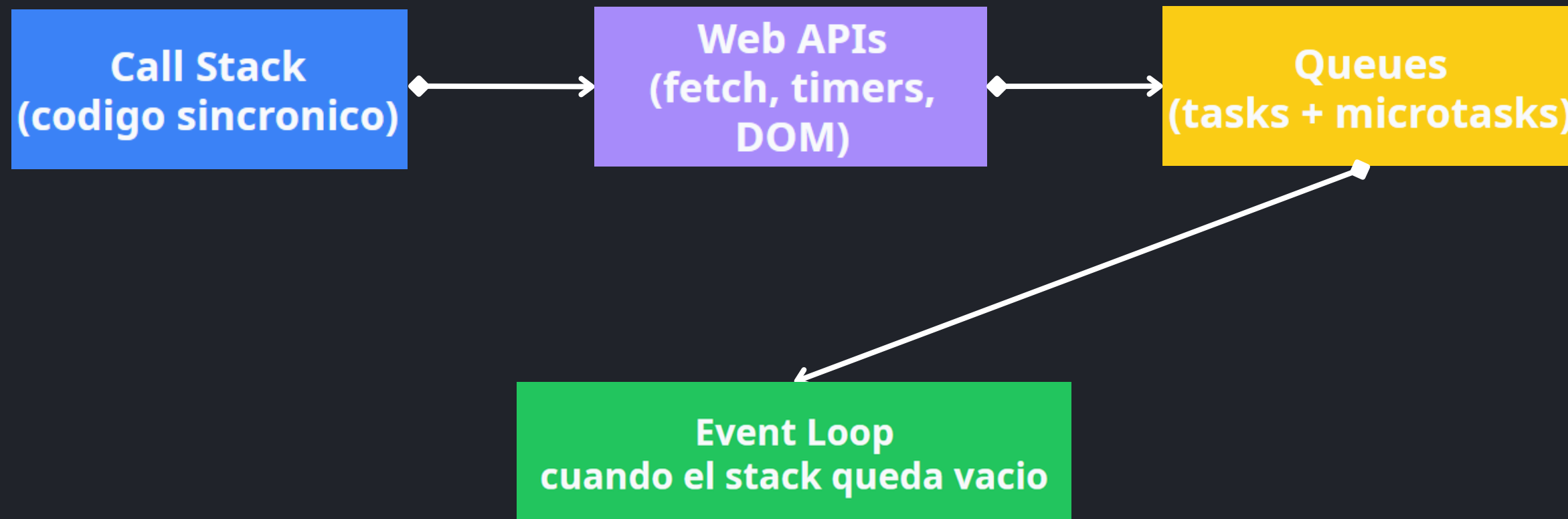
Al hacerlo síncrono, el Browser se congela. Durante esos 3 segundos:

- Ignora clicks del usuarios.
- Detiene el repintado de la animación.
- No funciona el Scroll.
- Experiencia del usuario se ve afectada.

Asincronía surge para delegar esperas prolongadas y mantener el hilo principal operativo.
NO lo hace Multi-hilo

Modelo del browser

JS ejecuta el stack; el navegador espera por fuera; el event loop decide cuando vuelven los callbacks.



Regla Base:

- Primero termina lo sincrónico
- Después se vacían las microtarear
- Luego entra una tarea de la callback queue
- Se repite

Orden de Ejecución

Una promesa resuelta entra como microtarea; setTimeout entra como tarea.



```
console.log('A');  
  
setTimeout(() => console.log('B'), 0);  
  
Promise.resolve()  
  .then(() => console.log('C'));  
  
console.log('D');
```

Output:



Por qué?

- A y D son sincrónicos
- C va a Microtask queue
- B va a Callback/Task queue
- Microtasks tienen prioridad sobre tasks

Callbacks: funciones que viajan como parámetros

Sirven en el fondo para indicar que, cuando pase X, ejecuta esta función.

```
function cuandoTermine(callback) {  
  setTimeout(() => {  
    callback('datos listos');  
  }, 1000);  
}  
  
cuandoTermine((resultado) => {  
  console.log(resultado);  
}));
```

- No toda callback es asíncrona: `forEach` usa callback, pero corre ahora.
- Una callback asíncrona queda guardada para ejecutarse después
- El problema se materializa cuando se anidan muchas esperas.

SINC

`array.forEach(fn)`

ASINC

`setTimeout(fn, 1000)`

Callback hell

El problema real es el control de flujo, manejo de errores y lectura de código.

```
login(user, pass, (err, session) => {  
  if (err) return mostrarError(err);  
  cargarPerfil(session, (err, perfil) => {  
    if (err) return mostrarError(err);  
    cargarPermisos(perfil, (err, permisos) => {  
      if (err) return mostrarError(err);  
      render(perfil, permisos);  
    });  
  });  
});
```

Síntomas:

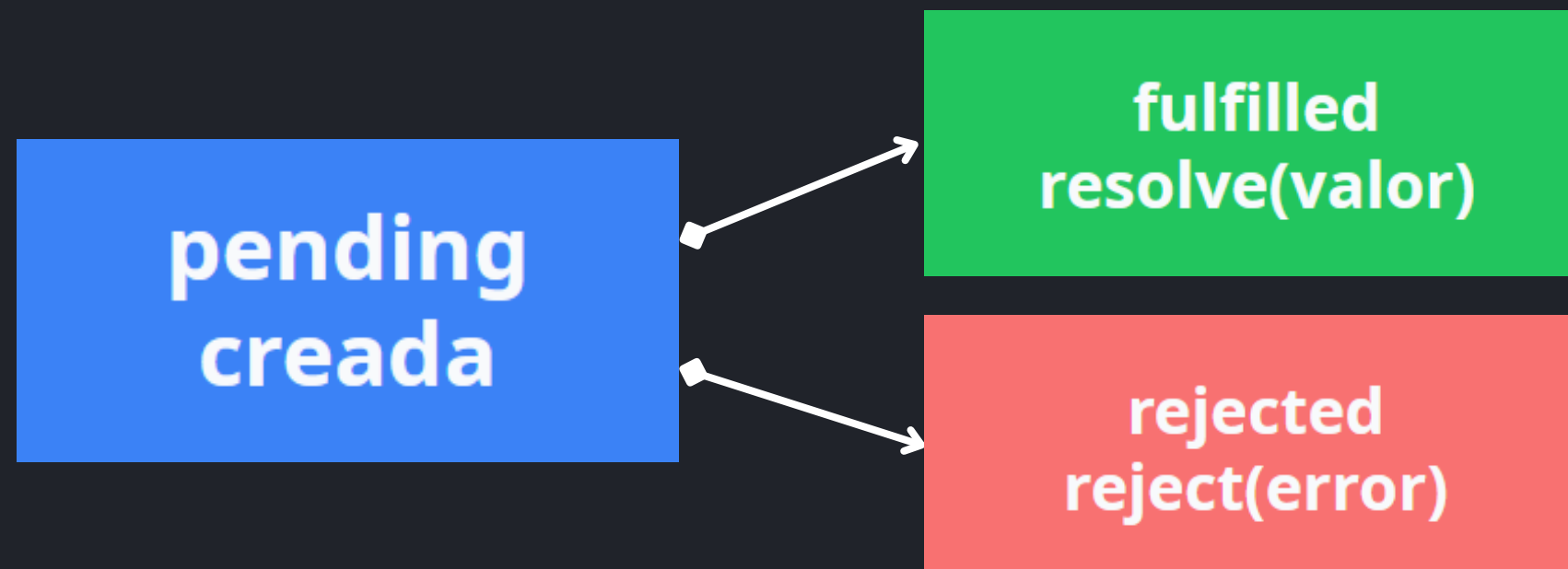
- Indentación crece a la derecha.
- Errores repetidos en cada nivel.
- Cuesta reutilizar cada paso.
- Difícil decidir qué corre en serie y que en paralelo.

Transición natural:

- Representar cada espera como un valor futuro, una Promise.

Promesas: valores futuros con estado

Una promesa representa una operación que todavía no termina.



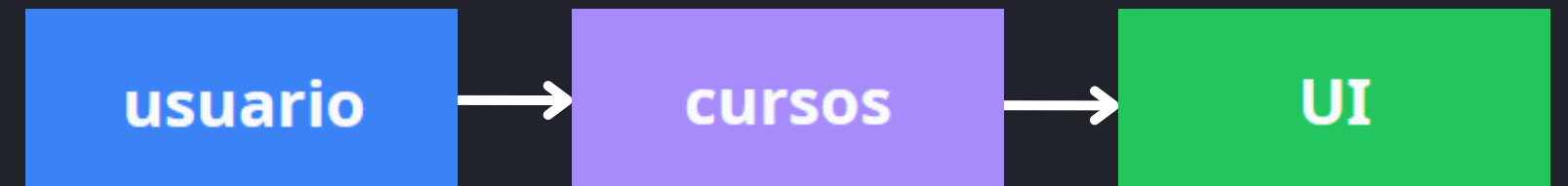
```
1  const promesa = new Promise((resolve, reject) => {  
2    setTimeout(() => {  
3      resolve('OK');  
4      // reject(new Error('fallo'));  
5    }, 1000);  
6  });
```

Importante: una promesa se resuelve una sola vez.

Encadenar promesas

Cada `.then` devuelve otra promesa. Eso permite pasar datos paso a paso.

```
1 obtenerUsuario()  
2   .then(usuario => obtenerCursos(usuario.id))  
3   .then(cursos => cursos.filter(c => c.activo))  
4   .then(cursosActivos => render(cursosActivos))  
5   .catch(error => mostrarError(error));
```



Bug frecuente

```
// Mal: se pierde la cadena  
.then(usuario => {  
  obtenerCursos(usuario.id);  
})
```

Falta `return` o devolver la promesa.

Errores en promesas: catch y finally

El “happy path” va por then; los errores bajan hasta el catch más cercano.



```
1 cargarDatos()  
2   .then(datos => pintar(datos))  
3   .catch(error => {  
4       console.error(error);  
5       mostrarMensaje('No se pudo cargar');  
6   })  
7   .finally(() => ocultarSpinner());
```

Patrón:

- then: uso del resultado.
- catch: recuperación o mensaje de error.
- finally: limpieza que debe ocurrir siempre
- Hay que manejar los errores explícitamente

catch no es solo para que el código no explote, también define lo que ve el usuario

async/await: promesas con sintaxis sincrónica

await pausa esa función async, no congela todo el código Javascript.

```
1  async function cargarCursos() {
2      try {
3          mostrarSpinner();
4          const usuario = await obtenerUsuario();
5          const cursos = await obtenerCursos(usuario.id);
6          render(cursos);
7      } catch (error) {
8          mostrarError(error);
9      } finally {
10         ocultarSpinner();
11     }
12 }
13
14
```

Idea:

- async function siempre devuelve una Promise.
- await espera una Promise dentro de esa función.
- try/catch reemplaza al .catch() en lectura.
- El resto de la página sigue viva.

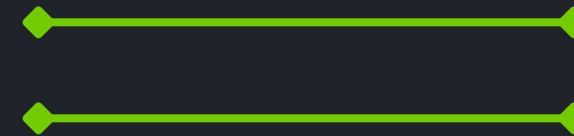
Serie vs paralelo

No todo await debe esperar al anterior. A veces puede ser conveniente lanzar varias promesas juntas.

```
1 // Serie: espera una, luego la otra
2 const perfil = await fetchPerfil();
3 const notif = await fetchNotificaciones();
```



```
1 // Paralelo: ambas parten al mismo tiempo
2 const [perfil, notif] = await Promise.all([
3     fetchPerfil(),
4     fetchNotificaciones()
5 ]);
```



Serie suma tiempos de ejecución. La versión paralela espera al más lento.
Promise.all falla si falla cualquiera.

fetch realista: respuesta HTTP no es JSON automáticamente

```
1  async function cargarUsuarios() {
2    const res = await fetch('/api/usuarios');
3
4    if (!res.ok) {
5      throw new Error(`HTTP ${res.status}`);
6    }
7
8    const usuarios = await res.json();
9    return usuarios;
10 }
```

Dos esperas distintas

1. await fetch()



2. await res.json()

Error común: creer que fetch devuelve directamente el arreglo.

DOM + async: patrón de botón con loading

Los eventos del DOM son el puente natural entre usuario, fetch y render.

```
1 btn.addEventListener('click', async () => {
2   btn.disabled = true;
3   estado.innerText = 'Cargando...';
4
5   try {
6     const data = await cargarUsuarios();
7     lista.innerHTML = data.map(u => '<li>' + u.name + '</li>').join('');
8     estado.innerText = 'Listo';
9   } catch (e) {
10    estado.innerText = 'Error al cargar';
11  } finally {
12    btn.disabled = false;
13  }
14 });
```

- Evento DOM: click.
- Web API: fetch
- Promise: resultado futuro.
- async/await: lectura lineal.
- finally: UI consistente.

Actividad

Punto 1: Deben convertir un Callback en una Promise

Punto 2: Usar async / await + fetch y deben controlar los errores.

Punto 3: Tienen que implementar código en serie y en paralelo

Entreguen archico asincronia.js en Canvas y respondan la pregunta.

```
<!--IIC2513-->
```

JS en Profundidad

{

```
<Tema="Asincronía & Event Loop"/>
```

```
<Por="Vicente Ignacio Soto González"/>
```

}

